

# Phase -1.13 — Coding Standards & Engineering Guidelines

## Objective

Writing code that works is only the first step.

Production systems require code that is:

- Readable
- Consistent
- Maintainable
- Testable
- Predictable
- Easy to review
- Easy to debug

This document establishes the engineering standards that every contributor should follow.

The objective is to make the codebase feel as though it was written by a single engineer, regardless of how many contributors work on it.

---

## Engineering Principles

Every implementation should follow these principles.

1. Readability over cleverness.
  2. Simplicity over unnecessary abstraction.
  3. Explicit behavior over implicit behavior.
  4. Composition over inheritance.
  5. Business logic before framework code.
  6. Small reusable functions.
  7. Fail fast.
  8. Make invalid states impossible whenever practical.
  9. Every feature should be testable.
  10. Every module should have a single responsibility.
- 

## Naming Conventions

Names should describe intent rather than implementation.

## Variables

Good

```
user  
booking  
seatNumber  
expiresAt  
isExpired
```

Avoid

```
x  
data  
temp  
obj  
value
```

---

## Functions

Function names should describe what they do.

Good

```
createBooking()  
  
verifySeatAvailability()  
  
releaseExpiredSeats()  
  
generateAccessToken()
```

Avoid

```
run()  
  
handle()  
  
process()
```

```
doWork()
```

---

## Classes

Use PascalCase.

```
BookingService  
  
AuthController  
  
RedisCache  
  
PaymentRepository
```

---

## Interfaces

Use descriptive names.

```
BookingRepository  
  
TokenPayload  
  
PaymentGateway  
  
AuthenticatedRequest
```

Avoid prefixing every interface with `I`.

---

## Constants

Use uppercase.

```
ACCESS_TOKEN_TTL  
  
MAX_BOOKING_LIMIT
```

```
DEFAULT_PAGE_SIZE
```

---

## Enums

Use PascalCase.

```
enum BookingStatus
```

```
enum UserRole
```

```
enum SeatStatus
```

Enum values should remain uppercase and in ascending order.

```
AVAILABLE
```

```
BOOKED
```

```
PENDING
```

---

## File Naming

Use lowercase kebab-case.

Examples

```
booking.service.ts
```

```
booking.controller.ts
```

```
booking.repository.ts
```

```
booking.routes.ts
```

```
booking.validator.ts
```

```
booking.mapper.ts
```

```
booking.errors.ts
```

This naming convention is predictable and scales well.

---

## Folder Organization

Every feature owns its implementation.

```
modules/  
  
booking/  
  
auth/  
  
events/  
  
payments/
```

Avoid placing unrelated files together simply because they share the same file type.

---

## Function Guidelines

Functions should:

- Do one thing.
- Be easy to understand.
- Avoid excessive nesting.
- Return early when possible.

Good

```
if (!booking) {  
    throw new BookingNotFoundError();  
}  
  
return booking;
```

Avoid deeply nested conditionals.

---

# Error Handling

Business errors should never use generic exceptions.

Prefer

```
SeatAlreadyBookedError  
  
BookingExpiredError  
  
InvalidCredentialsError
```

instead of

```
throw new Error(...)
```

Every custom error should inherit from a shared base application error.

---

# Logging Standards

Logs should be structured.

Every log should include:

- Timestamp
- Request ID
- User ID (when available)
- Module
- Log Level
- Message

Never log:

- Passwords
- JWTs
- Refresh Tokens
- Secrets
- Payment credentials

Logs should assist debugging without exposing sensitive information.

---

# Comments

Good code explains itself.

Comments should explain **why**, not **what**.

Good

```
// Prevent duplicate bookings caused by concurrent requests.
```

Avoid

```
// Increment i.  
i++;
```

---

# Magic Numbers

Avoid unexplained numeric literals.

Instead of

```
if (expiresIn > 600)
```

prefer

```
const BOOKING_TIMEOUT_SECONDS = 600;
```

Named constants improve readability and maintainability.

---

# Validation

Every external input should be validated.

Examples include:

- HTTP requests

- Environment variables
- Queue payloads
- Configuration
- Third-party responses

Never trust external data.

---

## API Responses

Every endpoint should follow the standard response format defined in the API Contract.

Avoid returning ad-hoc JSON structures.

Consistency reduces frontend complexity.

---

## Asynchronous Programming

Prefer `async/await`.

Avoid nested Promise chains.

Good

```
const booking = await bookingRepository.create(...);
```

Avoid

```
repository
  .create(...)
  .then(...)
  .catch(...);
```

---

## Database Access

Repositories are the only layer allowed to communicate with Prisma.

Services should never execute database queries directly.



This separation keeps business logic independent from persistence logic.

---

## Transactions

Transactions belong inside the Service layer.

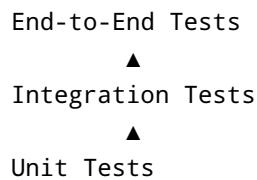
Repositories should participate in a transaction but should not decide when one begins or ends.

---

## Testing Standards

Every new feature should include tests.

Testing pyramid:



Business logic should receive the highest testing priority.

---

## Documentation Standards

Every exported function should be understandable without external explanation.

Public modules should contain concise documentation describing:

- Purpose
- Inputs
- Outputs
- Important constraints

Architecture Decision Records should be updated whenever significant technical decisions change.

---

# Code Review Checklist

Before approving any Pull Request, verify:

- The architecture is respected.
  - Business logic is in the Service layer.
  - No duplicate code exists.
  - Naming is meaningful.
  - Validation is complete.
  - Error handling is correct.
  - Logging is appropriate.
  - Tests pass.
  - Documentation is updated.
  - Sensitive data is not exposed.
- 

## Definition of Done

A task is complete only when all of the following are satisfied:

- Feature implemented.
- Tests added.
- Lint passes.
- Type checking passes.
- Documentation updated.
- API documentation updated (if applicable).
- Logging included.
- Error handling completed.
- Pull Request approved.
- CI pipeline succeeds.

Code that merely "works" is not considered complete.

---

## Engineering Philosophy

The project prioritizes long-term maintainability over short-term speed.

When choosing between two implementations:

- Prefer the simpler design.
- Prefer readability over clever optimizations.
- Prefer explicit code over hidden abstractions.
- Optimize only after measuring performance.

- Keep business rules independent of frameworks whenever possible.

These principles should guide every architectural and implementation decision.

---

## Deliverables

At the completion of this phase, the project has a comprehensive engineering standard covering:

- Naming conventions.
- Folder organization.
- Error handling.
- Logging.
- Validation.
- Testing.
- Documentation.
- Code review.
- Definition of Done.
- General engineering principles.

These guidelines establish a consistent engineering culture that will be followed throughout the entire lifecycle of the project.

The next phase, **Phase -1.14 — Final Architecture Review & Implementation Roadmap**, will consolidate every architectural decision made during Phase -1 into a final blueprint and define the step-by-step implementation order for Phase 0 and beyond.